TRUSTRIDE

Build plan and architecture

The stack, the modules, how they talk to each other, and the task list. Version 2, 2 October 2026: one mobile app, roles and permissions, partner companies planned for.

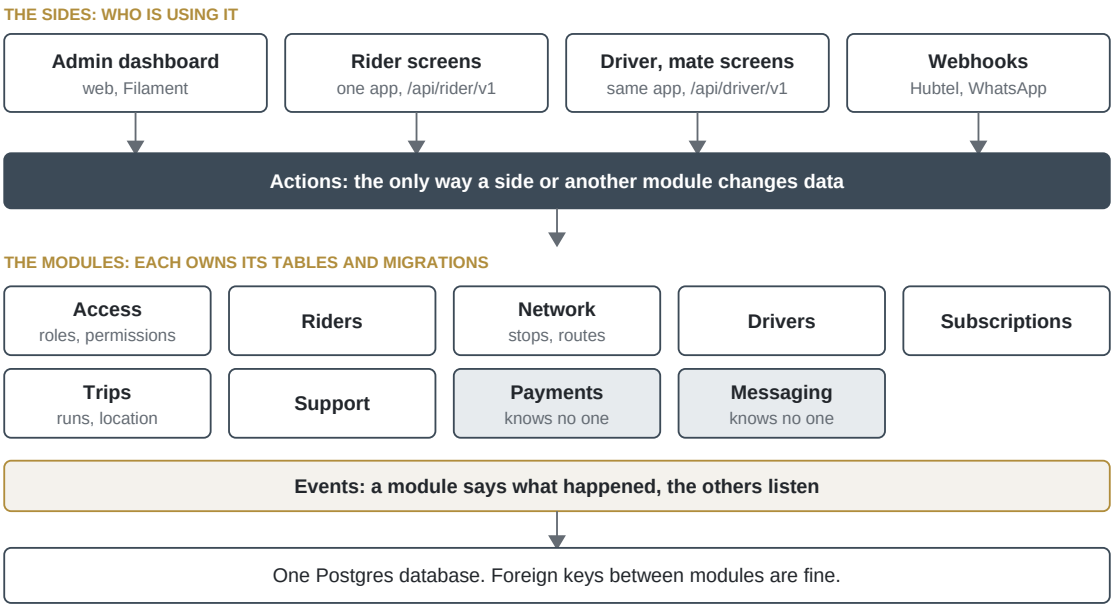
1. What we decided today

Question	Decision
Backend	Laravel on Postgres. One app, one database, one deploy.
Structure	Modular monolith. Each module has its own folder and its own migrations. Foreign keys between modules are allowed. Modules don't need to run alone.
Sides	Admin dashboard on the web. One mobile app for riders, drivers and mates. It opens the right screens for the account's roles.
Mobile	React Native with Expo. Android first. You've used neither Expo nor Flutter, so I picked. Reasons are in section 6.
Driver side	Driver and mate screens with live location, sent from both phones. This reopens something the agreed model dropped.
Roles	Configurable. One account can hold several, so a mate can also be a rider.
Permissions	Two scopes, platform and app, each bundled into groups. A third scope for partner companies comes later and is planned for now.
Unchanged	TrustRide never touches fare money. Fixed stops. No on-demand.

> **On the driver app.** I advised against it for v1 and you chose it anyway. That's recorded here, not reopened. The cost is real though. Background location on cheap Android phones is the hardest thing in this build, and by my estimate it adds three to four weeks. So the plan puts it last. The pilot can start with the mate reporting by phone while the driver screens get built. This also means agreed-model.pdf is now out of date on tracking.

2. The shape

Two kinds of folders. Modules hold the business. Sides hold the entry points. A side never contains a business rule. It checks who's calling, calls a module, and shapes the reply.



Module	What it owns
Access	Staff accounts, mobile accounts, roles, permissions, permission groups, organisations. New.
Riders	Riders, applications, verification, each rider's preferred message channel
Network	Stops, routes, the order of stops on a route
Drivers	Drivers, vehicles, vetting checks, overrides with who and why, assessments, commission
Subscriptions	Plans, subscriptions, seat allocation, the no-show policy
Trips	Each day's run, pickups, location pings, share links. New, because of the driver app.
Support	The WhatsApp queue and incidents
Payments	Hubtel charges, callbacks, receipts
Messaging	Event log, rules, templates, channels, delivery log

3. How modules talk

Three rules. Everything else follows from them.

Rule 1. Read anything

Any module can read another module's tables through relationships and joins. That's what keeps the console simple. "Today's riders per driver" is one query across four modules.

Rule 2. Write only your own tables

If Subscriptions needs a charge created, it doesn't touch the charges table. It calls an Action that Payments provides. Each module has an `Actions` folder, and that folder is its front door. Use an Action when you need the answer now.

Rule 3. Say what happened

When a module finishes something others might care about, it fires an event. It doesn't know who's listening. Use an event when you're telling, not asking.

Who is allowed to depend on whom

Module	May use
Access	Nothing. Every module below may use Access.
Network	Nothing
Riders	Network
Drivers	Network
Subscriptions	Riders, Network, Drivers, Payments
Trips	Subscriptions, Riders, Drivers, Network
Support	Riders, Drivers, Trips
Payments	Nothing
Messaging	Nothing

Payments and Messaging know nothing about riders or drivers. Payments charges "a thing with a type and an id" and reports back. Messaging receives an event name, some data, and a list of phone numbers with preferred channels. That's cheap to do, and it means those two could move into your next product later.

How Messaging hears everything without knowing anyone

Every event follows one small shared shape:

```
name()        e.g. subscription.activated
payload()     the data a template can use
recipients()  who it concerns: role, phone, preferred channel
```

One listener in Messaging catches all of them. It writes the event to the log, finds the rules an admin has set for that event name, and queues the sends. A new message is a new rule row. A new event is one new class.

Worked example: a rider pays and her seat goes live

1. The rider app posts her plan and stop to the rider API.
2. Subscriptions creates a pending subscription and calls the Payments action `StartCharge`.
3. Payments asks Hubtel to prompt her phone and saves the charge as pending.
4. Hubtel calls the webhook. Payments marks the charge paid and fires `payment.succeeded`.
5. Subscriptions hears it, runs `ActivateSubscription`, assigns her seat, and fires `subscription.activated`.
6. Messaging hears that, finds the rules, and sends her receipt on her preferred channel.

An operator who activates a subscription by hand in the console runs the same `ActivateSubscription`. Same event. Same messages.

The first set of events

Event	Fired by	Who listens
application.approved	Riders	Messaging
payment.succeeded, payment.failed	Payments	Subscriptions, Messaging
subscription.activated	Subscriptions	Trips, Messaging
subscription.expiring	Subscriptions, from a daily job	Messaging
trip.started	Trips	Messaging

Event	Fired by	Who listens
trip.delayed	Trips	Support, Messaging
rider.picked_up, rider.missed	Trips	Subscriptions for the no-show policy, Messaging
incident.raised	Support	Messaging

Four habits that keep this honest

- Events fire only after the database transaction commits. Nobody gets a receipt for a payment that rolled back.
- Listeners run on the queue and must be safe to run twice. Hubtel can call the webhook twice for one payment.
- No module calls save on another module's model. It calls the Action.
- An automated test fails the build if a module uses something outside the table above. Pest's architecture tests do this in a few lines.

4. Accounts, roles and permissions

Two account tables. Staff log in on the web with email and password. Everyone on mobile shares one account type and logs in with a phone number and SMS code. Login returns the account's roles and permissions, and the dashboard menu and the app's home screen are built from that list.

Scope	Who	Example permissions	When
Platform	TrustRide staff on the dashboard	drivers.vet, subscriptions.activate, roles.manage	v1
App	Riders, drivers and mates on mobile	trips.mark_pickup, subscription.renew, trip.share	v1
Partner	An outside company's admins, seeing only their own drivers and vehicles	Not defined yet	Later

- **Every role and permission carries its scope.** A platform role can't be given an app permission, and a mobile token can't pass a platform check.
- **Modules declare their own permissions**, next to their migrations. Access stores them.
- **A group is a named bundle**, such as "Driver vetting". An admin ticks a whole group or single permissions inside it.
- **A role keeps the group, not a copy.** A permission added to a group later reaches every role that has the group. This is my default. Say if you'd prefer copies.
- **Code checks permissions only.** Groups are for assigning and display, so regrouping never breaks a check.
- **Permissions don't replace ownership.** A rider who can view subscriptions sees only her own. Each module's policies enforce that.
- **Mobile screens are written in code.** A new role can combine existing screens. It can't create new ones.
- **The server checks every request.** Hiding a menu item is for convenience.

Planning for partner companies

I read your answer as: outside companies, such as a fleet owner, will get their own admins later. So v1 does one cheap thing now. Drivers, vehicles, routes and staff accounts each carry an organisation id, and TrustRide is organisation 1. The partner scope itself isn't built in v1. When it's needed, it's a filter and a set of

roles, not a rewrite of the data. Riders stay TrustRide's customers. If I read your answer wrong, this is one column to drop.

5. Live location, done cheaply

- **The driver's phone and the mate's phone both send a position every 15 seconds** while a trip is running. If the network drops, each stores them and sends the batch when it's back.
- **The server keeps the newest position per trip** in one row, whichever phone it came from, plus a history table. If one phone dies, the van stays on the map.
- **The mate marks pickups.** The driver is driving.
- **The rider app asks for the position every 15 seconds** while she's waiting. Plain requests, no live socket server. Twelve riders on one van is 48 requests a minute. That's nothing.
- **Show "3 stops away", not minutes.** Stops away comes from the route's stop order and costs nothing. Minutes need a paid routing service.
- **The share link is a web page** with a random token. It shows the same position and stops working when the trip ends.
- **The mate by phone stays as the fallback.** Operators can still set a trip's status by hand when the app is off.

> **The hard part.** Phones like Tecno, Infinix and itel shut down background apps to save battery. The app needs a permanent notification while a trip runs, and battery saving switched off for it on each phone. Google Play also asks you to justify background location before it approves the app. So the first driver task is a test on the drivers' and mates' real phones for a full morning run. If that fails, we learn it in three days, not three weeks.

Two things I haven't settled:

- **The map.** I haven't checked Google's current map prices for mobile apps. MapLibre with OpenStreetMap is the free option. Check before choosing.
- **How long location history is kept.** It's personal data about the driver and the mate. It needs a number of days, same as the rider retention question.

6. Why Expo, and what it costs

- **The framework is free and open source.** You only pay if you use Expo's cloud services past the free limits.
- **One codebase covers Android and iPhone.** Rider, driver and mate screens live in one app with one store listing.
- **It supports background location**, which the driver and mate screens need.
- **You can push a fix to phones without a store review**, for changes that don't touch native code.
- **Flutter would also work** and costs the same. The tiebreak is language. More developers know JavaScript than Dart, so help is easier to find later.

Item	Cost	Notes
Expo framework	Free	Open source.
Expo cloud builds	Free plan	15 Android and 15 iPhone builds a month. Building on your own laptop is free and unlimited.

Item	Cost	Notes
Expo updates to phones	Free to 1,000 users	The Starter plan is $19 a month after that. A good problem to have.
Google Play account	$25 once	New personal accounts must run a closed test with 12 testers for 14 days first. Your ten interested riders nearly fill that.
Apple account	$99 a year	Skip it until riders ask for iPhone.
Push notifications	Free	Firebase Cloud Messaging.
Hosting	Not checked	One small server and a managed database. Get a current price.
SMS, WhatsApp, Hubtel fees	Not checked	Per message and per payment. Get quotes with the merchant account.

> **One app, two releases.** Background location is declared for the whole app, not per role. So release 1 ships rider screens only, with no location permission, and goes through the normal review. Release 2 adds the driver and mate screens and background location as an update.

At pilot size the fixed software bill is $25 once. The running costs are hosting and messages, and messages are the one to watch.

7. Build order

Times are my estimates for one developer working full time. Treat them as rough.

Phase	What	Time	Waiting on
1	Foundations, including Access	1 to 2 weeks	
2	Core modules and the console	2 to 3 weeks	
3	Payments and subscriptions	1 to 2 weeks	Company registration, for a live Hubtel account. Build on the sandbox meanwhile.
4	Messaging	1 to 2 weeks	WhatsApp Business verification
5	Mobile app, release 1: rider screens	2 to 3 weeks	Play Store account and closed test
	Pilot can start here, with the mate on the phone		
6	Support and incidents	1 week	
7	Release 2: driver and mate screens, live location	3 to 4 weeks	Play Store background location review
8	Pilot hardening	1 week	
	Total	**12 to 18 weeks**	7 to 12 weeks to reach a pilot without the driver screens.

> **Against your three months.** The pilot fits, at 7 to 12 weeks. The driver and mate release is what pushes the full build past 13 weeks if the estimates run long. Starting the pilot before that release is how you keep the date. One app saves about a week of setup in phase 7. The mate role and two phones sending location use it up, so that estimate hasn't moved.

8. Tasks

Phase 1. Foundations

- [ ] **F1** Write the domain model: every entity, its fields, and the path from a rider signing up to a driver being paid at month end.
- [ ] **F2** New Laravel app on Postgres. Modules folder, one service provider per module, each loading its own migrations and routes.
- [ ] **F3** Access module: staff accounts and mobile accounts. Phone number and SMS code on mobile. Login returns roles and permissions.
- [ ] **F4** Permissions in two scopes, platform and app, declared by each module and bundled into groups. Roles editable in the console.
- [ ] **F5** Organisation id on drivers, vehicles, routes and staff accounts. TrustRide is organisation 1.
- [ ] **F6** The shared event shape, the Messaging catch-all listener as a stub, and the dependency test.
- [ ] **F7** Filament panel with its menu built from permissions.
- [ ] **F8** Deploy pipeline and daily database backups. Restore one backup to prove it works.
- [ ] **F9** Start the slow paperwork: Hubtel merchant account, WhatsApp Business verification, Play developer account, Data Protection Commission registration.

Phase 2. Core modules and the console

- [ ] **C1** Network: stops, routes, ordered stops on a route.
- [ ] **C2** Drivers: driver, vehicle, vetting checklist. An override records who approved it and why.
- [ ] **C3** Riders: application, verification, approve or reject with a reason.
- [ ] **C4** Subscriptions: the three plans, seats per driver, the no-show policy as one field per plan.
- [ ] **C5** Console: today's riders per driver, and capacity per route and time.
- [ ] **C6** Drivers: monthly assessment and commission record.

Phase 3. Payments and subscriptions

- [ ] **P1** Payments module against the Hubtel sandbox: start a charge, receive the callback, issue a receipt.
- [ ] **P2** Make the callback safe to receive twice.
- [ ] **P3** Subscription activates on payment.succeeded. Manual activation in the console uses the same Action.
- [ ] **P4** Daily job that expires subscriptions and fires subscription.expiring three days ahead.
- [ ] **P5** Switch to live once the merchant account exists.

Phase 4. Messaging

- [ ] **M1** Event log and the real catch-all listener.
- [ ] **M2** Rules: event, audience, channel, template. Editable in the console.
- [ ] **M3** Channels: SMS, WhatsApp, push. Fall back when the preferred one fails.
- [ ] **M4** Delivery log operators can read.
- [ ] **M5** An operator can message every rider on a driver's run in one go.

Phase 5. Mobile app, release 1: rider screens

☐	**R1**	Expo project. One app. The home screen follows the account's roles, with a switcher for accounts that hold more than one.
☐	**R2**	Sign up, apply, choose a stop and a time.
☐	**R3**	Pay, then see the receipt: car number, colour, driver, stop, time.
☐	**R4**	Home screen: her seat, today's status, renew.
☐	**R5**	Push notifications and her channel preference.
☐	**R6**	Delete my data. Built and tested, not promised.
☐	**R7**	Release 1 has no location permission. Play Store closed test with the interested riders.

Phase 6. Support and incidents

☐	**S1**	WhatsApp messages land in a queue in the console. Assign, reply, hand over.
☐	**S2**	In-app alert from rider to operator.
☐	**S3**	Trip share link page.
☐	**S4**	Incident record with a timeline.

Phase 7. Release 2: driver and mate screens, live location

☐	**D1**	Test first. Background location on the drivers' and mates' real phones for a full morning run. Go or stop on the evidence.
☐	**D2**	Trips module: the day's run, start and end, pickups.
☐	**D3**	Driver and mate screens: today's riders by stop. The mate marks picked up or missed.
☐	**D4**	Location pings from both phones with offline batching. The server keeps the newest per trip.
☐	**D5**	Rider screens: map and "stops away".
☐	**D6**	Share link shows the live position.
☐	**D7**	Console live view. Operators can still set status by hand.
☐	**D8**	Driver sees his commission and assessment result.
☐	**D9**	Play Store background location declaration, then ship release 2.

Phase 8. Pilot hardening

☐	**H1**	Retention job and deletion path, tested on real records.
☐	**H2**	The report for whichever number tells you it's working.
☐	**H3**	One full week with both drivers and test riders before charging the public.

9. Open questions that block tasks

Open question	Blocks
The driver's fixed monthly commission	C6, D8, and whether the numbers work at all
Which pickup area and destination is the pilot	C1, R7

Open question	Blocks
The launch default for no-shows	C4
Was dropping the one-week plan deliberate	C4
What the monthly driver assessment measures	C6
How many days before a rider counts as no longer interested	H1
The number that says it's working, and what would make you stop	H2
How many days location history is kept	D4
New: does a role keep the group or a copy of its permissions	F4
New: what a partner company's admin may see and do	Nothing in v1

None of these stop Phase 1. F1, the domain model, is still the next thing to write.